UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
CURSO: ORGANIZACIÓN DE LENGUAJES Y COMPILADORES
CATEDRÁTICO: ING. MOISES GRANADOS

PROYECTO 1: GENERADOR DE FORMULARIOS
MANUAL TÉCNICO

ESTUDIANTE	**CARNÉ:**
Cristian Alejandro Roldán López	202147280

QUETZALTENANGO, 24 DE MARZO 2,026

Tecnologías usadas:

Sistema operativo: Ubuntu 24.04 LTS
IDE: Intelliji Idea versión Ultimate
Plugin: Android Studio
Framework: Jetpack Compose
Java: Versión 17
minSdk : 24
compileSdk: 36

Servidor: Apache TomCat 10

```
package
com.example.proyecto1_compi1.modelo.question;

import java.util.ArrayList;

public abstract class QuestionModel {

   protected String label;
   protected int    width;
   protected int    height;
   protected ArrayList<Object> styles;

   public QuestionModel() {
       styles = new ArrayList<>();
   }

   public void addProperty(PropertyItem prop) {

       if (prop == null || prop.getKey() == null)
return;

       switch (prop.getKey()) {

           case "label":
               if (prop.getValue() != null) {
                   label =
prop.getValue().toString()
                           .replace("\"", "")
                           .trim();
               }
               break;

           case "width":
               width = toInt(prop.getValue());
               break;
```

```
            case "height":
                height = toInt(prop.getValue());
                break;

            case "styles":
                if (prop.getValue() instanceof
ArrayList) {
                    styles = (ArrayList<Object>)
prop.getValue();
                }
                break;
        }
    }


    private int toInt(Object value) {
        if (value == null) return 0;
        if (value instanceof Integer) return
(Integer) value;
        if (value instanceof Double)  return
((Double) value).intValue();
        if (value instanceof Float)   return
((Float) value).intValue();
        if (value instanceof Long)    return
((Long) value).intValue();
        if (value instanceof Number)  return
((Number) value).intValue();
        if (value instanceof String) {
            try {
                return (int)
Double.parseDouble((String) value);
            } catch (NumberFormatException e) {
                return 0;
            }
```

```
        }
        return 0;
    }

    public String getLabel()              { return
label;  }
    public int    getWidth()              { return
width;  }
    public int    getHeight()             { return
height; }
    public ArrayList<Object> getStyles() { return
styles; }
}
```

```
package com.example.proyecto1_compi1.modelo.table;

import
com.example.proyecto1_compi1.analizador.form.Parse
r;
import
com.example.proyecto1_compi1.modelo.color_style.*;
import
com.example.proyecto1_compi1.modelo.question.Prope
rtyItem;

import java.util.ArrayList;

public class TableModel {

    private int width;
    private int height;
    private int pointX;
    private int pointY;
```

```
    private ArrayList<ArrayList<Object>> elements =
new ArrayList<>();
    private ArrayList<Object> styles = new
ArrayList<>();
    private Parser parser = new Parser();

    public TableModel(ArrayList<PropertyItem>
props) {
        for (PropertyItem prop : props) {
            addProperty(prop);
        }
    }

    public void addProperty(PropertyItem prop){
        switch(prop.getKey()){

            case "width":
                width = toInt(prop.getValue());
                break;

            case "height":
                height = toInt(prop.getValue());
                break;

            case "pointX":
                pointX = toInt(prop.getValue());
                break;

            case "pointY":
                pointY = toInt(prop.getValue());
                break;

            case "elements":
```

```
                elements =
(ArrayList<ArrayList<Object>>) prop.getValue();
                break;

            case "styles":
                if (prop.getValue() instanceof
ArrayList) {
                    ArrayList<Object> rawStyles =
(ArrayList<Object>) prop.getValue();
                    ArrayList<Object>
evaluatedStyles = new ArrayList<>();

                    for (Object style : rawStyles)
{
                        if (style instanceof
RgbStyleWithWildcard) {
                            RgbColor color =
((RgbStyleWithWildcard)
style).evaluar(parser.semantico);
                            evaluatedStyles.add(new
ColorStyle(color));
                        } else if (style instanceof
BackgroundStyleWithWildcard) {
                            RgbColor color =
((BackgroundStyleWithWildcard)
style).evaluar(parser.semantico);
                            evaluatedStyles.add(new
BackgroundStyle(color));
                        } else if (style instanceof
HslStyleWithWildcard) {
                            HslColor color =
((HslStyleWithWildcard)
style).evaluar(parser.semantico);
                            evaluatedStyles.add(new
ColorStyle(color));
```

```
                        } else if (style instanceof
HslBackgroundStyleWithWildcard) {
                            HslColor color =
((HslBackgroundStyleWithWildcard)
style).evaluar(parser.semantico);
                            evaluatedStyles.add(new
BackgroundStyle(color));
                        } else {

evaluatedStyles.add(style);
                        }
                    }

                    styles = evaluatedStyles;
                }
                break;
        }
    }

    private int toInt(Object value) {
        if (value == null) return 0;
        if (value instanceof Integer) return
(Integer) value;
        if (value instanceof Double) return
((Double) value).intValue();
        if (value instanceof Float) return ((Float)
value).intValue();
        if (value instanceof Long) return ((Long)
value).intValue();
        if (value instanceof Number) return
((Number) value).intValue();
        if (value instanceof String) {
            try {
                return (int)
Double.parseDouble((String) value);
```

```
            } catch (NumberFormatException e) {
                return 0;
            }
        }
        return 0;
    }

    public int getWidth() {
        return width;
    }

    public int getHeight() {
        return height;
    }

    public int getPointX() {
        return pointX;
    }

    public int getPointY() {
        return pointY;
    }

    public ArrayList<Object> getStyles() {
        return styles;
    }

    public ArrayList<ArrayList<Object>>
getElements() {
        return elements;
    }
}
```

```
package
com.example.proyecto1_compi1.modelo.question;
```

```
import
com.example.proyecto1_compi1.analizador.semantico.
AnalizadorSemantico;
import
com.example.proyecto1_compi1.modelo.color_style.*;

import java.util.ArrayList;
import java.util.List;

/**
* SpecialQuestion — pregunta con comodines que se
resuelven en .draw()
*
* MODELO CORRECTO DE COMODINES:
* ═══════════════════════════════
* Los comodines son posicionales y globales, no
por propiedad.
* Si una special tiene:
*   label: "ID " + ? + ": texto " + ?
*   correct: ?
* El label llega como "ID ?: texto ?" (2
comodines) y correct = "?"
* → totalWildcards = 3
* → draw(arg0, arg1, arg2) reemplaza en orden: 1er
?, 2do ?, 3er ?
*
* ORDEN DE RESOLUCIÓN:
* ═══════════════════
* Se recorre prop por prop en orden de
declaración.
* Dentro de cada prop se reemplazan los "?" de
izquierda a derecha.
* Cada "?" consume un argumento del draw().
*
```

```
* EXCLUSIONES:
* ______________
* "styles" y "options_poke" NO tienen comodines
resolubles
* (los comodines en RGB/HSL de styles son para
RgbStyleWithWildcard,
*  no para el sistema de draw).
* "options" SÍ puede tener comodines (opciones
dinámicas).
* "correct" SÍ puede tener comodines (-999 =
comodín en int_list).
*/
public class SpecialQuestion {

   private String              name;
   private String              type;
   private List<PropertyItem>  properties;
   private int                 totalWildcards; //
total global de "?" en todas las props
   private List<Object>        drawArguments = new
ArrayList<>();

   private static final AnalizadorSemantico EVAL =
new AnalizadorSemantico();

   //
____________________________________________________
_____________________________

   public SpecialQuestion(String name, String
type,
                          List<PropertyItem>
properties,
                          List<Integer>
wildcardIndicesIgnorado) {
       this.name       = name;
```

```
        this.type       = type;
        this.properties = new
ArrayList<>(properties);

        // Cuenta el total global de comodines en
todas las props
        this.totalWildcards =
contarTotalComodines(properties);

        System.out.println("[SpecialQuestion] '" +
name + "'" +
                " type=" + type +
                " totalWildcards=" + totalWildcards
+
                " props=" + properties.size());

        // Log detallado por prop
        for (int i = 0; i < properties.size(); i++)
{
            PropertyItem p = properties.get(i);
            int cnt =
contarComodinesEnValor(p.value);
            if (cnt > 0) {
                System.out.println("  prop[" + i +
"] key=" + p.key +
                        " value=" + p.value + "
wildcards=" + cnt);
            }
        }
    }

    //
____________________________________________________
_____________________________

    //  API PÚBLICA
```

```
    //
____________________________________________________________________________
    public int getWildcardCount() { return
totalWildcards; }

    public void setDrawArguments(List<Object> args)
{
        this.drawArguments = args != null ? new
ArrayList<>(args) : new ArrayList<>();
        System.out.println("[SpecialQuestion]
setDrawArguments(" +
                drawArguments.size() + "): " +
drawArguments);
    }

    //
____________________________________________________________________________
    //  CONTEO DE COMODINES
    //
____________________________________________________________________________

    /**
     * Cuenta el total de "?" en todas las
propiedades (excepto styles y options_poke).
     * Este número determina cuántos argumentos
necesita draw().
     */
    private int
contarTotalComodines(List<PropertyItem> props) {
        int total = 0;
        for (PropertyItem item : props) {
```

```
            if (esExcluidoDelConteo(item.key))
continue;
            total +=
contarComodinesEnValor(item.value);
        }
        return total;
    }

    /**
     * Cuenta cuántos "?" hay en un valor de forma
recursiva.
     * Un string "ID ?: texto ?" tiene 2 comodines.
     * Una lista ["?", "opcion", "?"] tiene 2
comodines.
     */
    private int contarComodinesEnValor(Object
value) {
        if (value == null) return 0;

        if (value instanceof String) {
            String s = (String) value;
            // Cuenta ocurrencias de "?" en el
string
            int count = 0;
            for (int i = 0; i < s.length(); i++) {
                if (s.charAt(i) == '?') count++;
            }
            return count;
        }

        if (value instanceof List) {
            int count = 0;
            for (Object item : (List<?>) value) {
                // En int_list, -999 es el marcador
de comodín
```

```
                if (item instanceof Integer &&
(Integer) item == -999) {
                    count++;
                } else {
                    count +=
contarComodinesEnValor(item);
                }
            }
            return count;
        }

        // Número, booleano, etc. → no tiene
comodín
        return 0;
    }

    /**
     * Las styles y options_poke se excluyen del
conteo principal.
     * Los comodines en styles (RGB/HSL) son
manejados por RgbStyleWithWildcard.
     * Los comodines en options_poke son parámetros
de la PokeAPI.
     */
    private boolean esExcluidoDelConteo(String key)
{
        return "styles".equals(key); //
options_poke SI cuenta wildcards
    }

    //
================================================================================
    //  RESOLUCIÓN DE COMODINES
```

```
    //
____________________________________________________________________________________________________

    /**
     * Resuelve todas las propiedades reemplazando
los "?" con los argumentos
     * de draw() en orden secuencial global.
     *
     * El índice de argumento se comparte entre
todas las propiedades:
     * si label consume los args 0 y 1, correct
consumirá el arg 2.
     */
    public List<PropertyItem>
getResolvedProperties() {
        int[] argCursor = {0};
        List<PropertyItem> resolved = new
ArrayList<>();

        for (PropertyItem item : properties) {
            if ("styles".equals(item.key)) {
                resolved.add(new
PropertyItem(item.key, item.value));
                continue;
            }

            if ("options_poke".equals(item.key)) {

                Object nuevoValor =
resolverValor(item.value, argCursor);
                resolved.add(new
PropertyItem(item.key, nuevoValor));
                continue;
            }
```

```
            Object nuevoValor =
resolverValor(item.value, argCursor);
            resolved.add(new PropertyItem(item.key,
nuevoValor));
        }

        return resolved;
    }


    private Object resolverValor(Object value,
int[] cursor) {
        if (value == null) return null;

        if (value instanceof String) {
            return resolverString((String) value,
cursor);
        }

        if (value instanceof List) {
            return resolverLista((List<?>) value,
cursor);
        }

        // Número, etc. → sin cambios
        return value;
    }

    private Object resolverString(String template,
int[] cursor) {
        if (!template.contains("?")) return
template;

        StringBuilder sb    = new StringBuilder();
```

```
        boolean esAritmetica = true; // asume
aritmética hasta que encuentre texto

        for (int i = 0; i < template.length(); i++)
{
            if (template.charAt(i) == '?' &&
cursor[0] < drawArguments.size()) {
                Object arg =
drawArguments.get(cursor[0]++);
                String argStr = argToString(arg);
                sb.append(argStr);
                // Si el arg no es numérico, no es
expresión aritmética
                if (!esNumerico(argStr))
esAritmetica = false;
            } else {
                char c = template.charAt(i);
                sb.append(c);
                // Si hay caracteres no
numéricos/operadores, no es aritmética pura
                if (!Character.isDigit(c) && c !=
'.' && c != '+' &&
                        c != '-' && c != '*' && c
!= '/' && c != ' ') {
                    esAritmetica = false;
                }
            }
        }

        String resultado = sb.toString();

        if (esAritmetica &&
resultado.matches(".*[+\\-*/].*")) {
            try {
```

```
                double val =
evaluarAritmetica(resultado);
                if (val == Math.floor(val) &&
!Double.isInfinite(val))
                    return (int) val;
                return val;
            } catch (Exception ignored) {}
        }

        return resultado;
    }

    private Object resolverLista(List<?> lista,
int[] cursor) {
        List<Object> nueva = new ArrayList<>();
        for (Object item : lista) {
            // -999 es el marcador de comodín en
int_list (correct)
            if (item instanceof Integer &&
(Integer) item == -999) {
                if (cursor[0] <
drawArguments.size()) {
                    Object arg =
drawArguments.get(cursor[0]++);
                    try {
                        nueva.add((int)
Double.parseDouble(arg.toString()));
                    } catch (Exception e) {
                        nueva.add(arg);
                    }
                } else {
                    nueva.add(-1);
                }
            } else if (item instanceof String &&
((String) item).contains("?")) {
```

```
                nueva.add(resolverString((String)
item, cursor));
            } else {
                nueva.add(item);
            }
        }
        return nueva;
    }


    public QuestionModel draw() {
        List<PropertyItem> resolved =
getResolvedProperties();
        System.out.println("[draw] type='" + type +
"' resolved props=" + resolved.size());

        switch (type.toUpperCase()) {
            case "OPENQUESTION": {
                OpenQuestion q = new
OpenQuestion();
                applyProps(q, resolved);
                System.out.println("[draw]
OpenQuestion label='" + q.getLabel() +
                        "' w=" + q.getWidth() + "
h=" + q.getHeight());
                return q;
            }
            case "DROPQUESTION": {
                DropQuestion q = new
DropQuestion();
                applyProps(q, resolved);
                System.out.println("[draw]
DropQuestion label='" + q.getLabel() +
                        "' opts=" +
q.getOptions());
```

```
                return q;
            }
            case "SELECTQUESTION": {
                SelectQuestion q = new
SelectQuestion();
                applyProps(q, resolved);
                System.out.println("[draw]
SelectQuestion label='" + q.getLabel() +
                        "' opts=" + q.getOptions()
+ " correct=" + q.getCorrect());
                return q;
            }
            case "MULTIPLEQUESTION": {
                MultipleQuestion q = new
MultipleQuestion();
                applyProps(q, resolved);
                System.out.println("[draw]
MultipleQuestion label='" + q.getLabel() +
                        "' opts=" + q.getOptions()
+ " correct=" + q.getCorrect());
                return q;
            }
            default:
                System.out.println("[draw] TIPO NO
RECONOCIDO: '" + type + "'");
                return null;
        }
    }

    private void applyProps(QuestionModel q,
List<PropertyItem> props) {
        for (PropertyItem prop : props) {
            if (prop == null || prop.key == null)
continue;
            if ("options_poke".equals(prop.key)) {
```

```
                q.addProperty(prop);
                continue;
            }
            System.out.println("[applyProps] " +
prop.key + " = " + prop.value);
            q.addProperty(prop);
        }
    }


    private String argToString(Object arg) {
        if (arg == null) return "";
        if (arg instanceof Double) {
            double d = (Double) arg;
            if (d == Math.floor(d) &&
!Double.isInfinite(d))
                return String.valueOf((long) d);
            return String.valueOf(d);
        }
        return arg.toString();
    }

    private boolean esNumerico(String s) {
        try { Double.parseDouble(s); return true; }
        catch (Exception e) { return false; }
    }


    private double evaluarAritmetica(String expr) {
        expr = expr.trim().replaceAll("\\s+", "");

        expr = aplicarOperador(expr, '^',
Math::pow);
        expr = aplicarOperador(expr, '*', (a, b) ->
a * b);
```

```
        expr = aplicarOperador(expr, '/', (a, b) ->
b != 0 ? a / b : 0);
        expr = aplicarOperador(expr, '+',
Double::sum);
        expr = aplicarOperador(expr, '-', (a, b) ->
a - b);

        return Double.parseDouble(expr.trim());
    }

    private String aplicarOperador(String expr,
char op,

java.util.function.BiFunction<Double, Double,
Double> func) {
        int idx = buscarOperador(expr, op);
        while (idx > 0) {
            int leftStart = idx - 1;
            while (leftStart >= 0 &&

(Character.isDigit(expr.charAt(leftStart)) ||
                                expr.charAt(leftStart)
== '.' ||
                                (leftStart == 0 &&
expr.charAt(leftStart) == '-'))) {
                leftStart--;
            }
            leftStart++;

            int rightEnd = idx + 1;
            if (rightEnd < expr.length() &&
expr.charAt(rightEnd) == '-') rightEnd++;
            while (rightEnd < expr.length() &&

(Character.isDigit(expr.charAt(rightEnd)) ||
```

```
                                expr.charAt(rightEnd)
== '.')) {
                rightEnd++;
            }

            try {
                double left   =
Double.parseDouble(expr.substring(leftStart,
idx));
                double right  =
Double.parseDouble(expr.substring(idx + 1,
rightEnd));
                double result = func.apply(left,
right);
                String resStr = (result ==
Math.floor(result) && !Double.isInfinite(result))
                        ? String.valueOf((long)
result)
                        : String.valueOf(result);
                expr = expr.substring(0, leftStart)
+ resStr + expr.substring(rightEnd);
                idx  = buscarOperador(expr, op);
            } catch (Exception e) {
                break;
            }
        }
        return expr;
    }

    private int buscarOperador(String expr, char
op) {
        for (int i = 1; i < expr.length(); i++) {
            if (expr.charAt(i) == op) {
                if (op == '-' && (expr.charAt(i -
1) == '*' || expr.charAt(i - 1) == '/'))
```

```
                    continue;
                return i;
            }
        }
        return -1;
    }

    public String              getName()
{ return name;            }
    public String              getType()
{ return type;            }
    public List<PropertyItem>  getProperties()
{ return properties;      }
    public List<Integer>       getWildcardIndices()
{ return new ArrayList<>(); } // legado
}
```

```
@RequiresApi(Build.VERSION_CODES.O)
fun generarPKMCompleto(
    nameForm:    String,
    autor:       String,
    descripcion: String,
    forms:       List<FormsModel>
): String {

    val fecha = LocalDate.now()
    val hora  = LocalTime.now().withNano(0)
    val elementos = mutableListOf<Any>()

    forms.forEach { form ->
        form.elements.forEach { el ->
            // Solo recolectar elementos
estructurales (no variables ni ciclos)
```

```
            recolectarElementoEstructural(el,
elementos)
        }
    }

    var totalSecciones = 0
    var totalPreguntas = 0
    var totalAbiertas  = 0
    var totalDrop      = 0
    var totalSelect    = 0
    var totalMultiple  = 0
    var totalTextos    = 0

    fun contarRecursivo(el: Any) {
        when (el) {
            is SectionsModel -> {
                totalSecciones++
                el.elements?.forEach { sub ->
contarRecursivo(sub) }
            }
            is OpenQuestion    -> {
totalPreguntas++; totalAbiertas++ }
            is SelectQuestion  -> {
totalPreguntas++; totalSelect++   }
            is DropQuestion    -> {
totalPreguntas++; totalDrop++     }
            is MultipleQuestion -> {
totalPreguntas++; totalMultiple++ }
            is TextModel       -> { totalTextos++ }
        }
    }
    elementos.forEach { contarRecursivo(it) }

    val contenido = StringBuilder()
```

```
    elementos.forEach { el ->
contenido.append(serializarElemento(el)) }

    return buildString {
        appendLine("###")
        appendLine("Author: $autor")
        appendLine("Fecha: $fecha")
        appendLine("Hora: $hora")
        appendLine("Descripcion: $descripcion")
        appendLine("Total de Secciones:
$totalSecciones")
        appendLine("Total de Preguntas:
$totalPreguntas")
        appendLine("    Abiertas: $totalAbiertas")
        appendLine("    Desplegables: $totalDrop")
        appendLine("    Seleccion: $totalSelect")
        appendLine("    Multiples: $totalMultiple")
        appendLine("    Textos (Abiertos):
$totalTextos")
        appendLine("###")
        appendLine()
        append(contenido.toString())
    }
}


private fun recolectarElementoEstructural(el:
Any?, dest: MutableList<Any>) {
    if (el == null) return

    when (el) {
        // Elementos que SÍ se guardan en PKM
        is OpenQuestion,
        is SelectQuestion,
        is DropQuestion,
```

```
        is MultipleQuestion,
        is TextModel,
        is TableModel -> {
            dest.add(el)
        }

        is SectionsModel -> {
            // Guardar la sección
            dest.add(el)
            el.elements?.forEach { sub ->
                recolectarElementoEstructural(sub,
dest)
            }
        }

        // Si es una pregunta genérica
        is QuestionModel -> dest.add(el)

        else -> {
        }
    }
}

fun serializarElemento(el: Any): String =
buildString {
    when (el) {
        is TextModel         ->
append(serializarTexto(el))
        is OpenQuestion      ->
append(serializarOpen(el))
        is SelectQuestion    ->
append(serializarSelect(el))
        is DropQuestion      ->
append(serializarDrop(el))
```

```
        is MultipleQuestion ->
append(serializarMultiple(el))
        is SectionsModel    ->
append(serializarSeccion(el))
        is TableModel       ->
append(serializarTabla(el))
    }
}

private fun serializarTexto(t: TextModel): String
= buildString {
    val label = cleanStr(t.content ?: "")
    if (!t.styles.isNullOrEmpty()) {

appendLine("<text=${t.width},${t.height},\"$label\
">")
        append(serializarEstilos(t.styles))
        appendLine("</text>")
    } else {

appendLine("<text=${t.width},${t.height},\"$label\
"/>")
    }
}

private fun serializarOpen(q: OpenQuestion):
String = buildString {
    val label = cleanStr(q.label ?: "")
    if (!q.styles.isNullOrEmpty()) {

appendLine("<open=${q.width},${q.height},\"$label\
">")
        append(serializarEstilos(q.styles))
        appendLine("</open>")
    } else {
```

```
appendLine("<open=${q.width},${q.height},\"$label\
"/>")
    }
}

private fun serializarSelect(q: SelectQuestion):
String = buildString {
    val label = cleanStr(q.label ?: "")
    val opts = q.options.joinToString(",") {
"\"${cleanStr(it)}\"" }
    val correct = q.correct.firstOrNull() ?: -1

    if (!q.styles.isNullOrEmpty()) {

appendLine("<select=${q.width},${q.height},\"$labe
l\",{$opts},$correct>")
        append(serializarEstilos(q.styles))
        appendLine("</select>")
    } else {

appendLine("<select=${q.width},${q.height},\"$labe
l\",{$opts},$correct/>")
    }
}

private fun serializarDrop(q: DropQuestion):
String = buildString {
    val label = cleanStr(q.label ?: "")
    val opts = q.options.joinToString(",") {
"\"${cleanStr(it)}\"" }
    val correct = q.correct.firstOrNull() ?: -1

    if (!q.styles.isNullOrEmpty()) {
```

```
appendLine("<drop=${q.width},${q.height},\"$label\
",{$opts},$correct>")
        append(serializarEstilos(q.styles))
        appendLine("</drop>")
    } else {

appendLine("<drop=${q.width},${q.height},\"$label\
",{$opts},$correct/>")
    }
}

private fun serializarMultiple(q:
MultipleQuestion): String = buildString {
    val label = cleanStr(q.label ?: "")
    val opts = q.options.joinToString(",") {
"\"${cleanStr(it)}\"" }
    val correct = q.correct.joinToString(",")

    if (!q.styles.isNullOrEmpty()) {

appendLine("<multiple=${q.width},${q.height},\"$la
bel\",{$opts},{$correct}>")
        append(serializarEstilos(q.styles))
        appendLine("</multiple>")
    } else {

appendLine("<multiple=${q.width},${q.height},\"$la
bel\",{$opts},{$correct}/>")
    }
}

private fun serializarSeccion(s: SectionsModel):
String = buildString {
```

```
appendLine("<section=${s.width},${s.height},${s.po
intX},${s.pointY},${s.orientation}>")

    if (!s.styles.isNullOrEmpty()) {
        append(serializarEstilos(s.styles))
    }

    appendLine("<content>")
    s.elements?.forEach { sub ->
        if (sub != null) {
            append(serializarElemento(sub))
        }
    }
    appendLine("</content>")
    appendLine("</section>")
    appendLine()
}

private fun serializarTabla(t: TableModel): String
= buildString {

appendLine("<table=${t.width},${t.height},${t.poin
tX},${t.pointY}>")

    if (!t.styles.isNullOrEmpty()) {
        append(serializarEstilos(t.styles))
    }

    appendLine("<content>")
    t.elements.forEach { row ->
        appendLine("<line>")
        (row as? List<*>)?.forEach { cell ->
            appendLine("<element>")
            if (cell != null) {
```

```
                append(serializarElemento(cell))
            }
            appendLine("</element>")
        }
        appendLine("</line>")
    }
    appendLine("</content>")
    appendLine("</table>")
    appendLine()
}

/**
* Serializa estilos - SOLO el contenido, no el
nombre de la clase
*/
fun serializarEstilos(styles: List<Any>?): String
{
    if (styles.isNullOrEmpty()) return ""
    return buildString {
        appendLine("<style>")
        styles.forEach { style ->
            when (style) {
                is ColorStyle -> {

appendLine("<color=${serializarColor(style.color)}
/>")
                }
                is BackgroundStyle -> {
                    appendLine("<background
color=${serializarColor(style.color)}/>")
                }
                is TextSizeStyle -> {
                    appendLine("<text
size=${style.size}>")
                }
```

```
                is FontStyle -> {
                    appendLine("<font
family=${style.font}/>")
                }
                is BorderStyle -> {

appendLine("<border,${style.width},${style.type},c
olor=${serializarColor(style.color)}/>")
                }
                is String -> {
                    // Formato esperado:
"SOLID:2:#FF3C43"
                    val parts = style.split(":")
                    if (parts.size >= 3) {
                        val tipo = parts[0]
                        val grosor = parts[1]
                        val color = parts[2]

appendLine("<border,$grosor,$tipo,color=$color/>")
                    } else {
                        appendLine("<!-- $style
-->")
                    }
                }
                else -> {

                }
            }
        }
        appendLine("</style>")
    }
}
```

```
fun serializarColor(color: ColorValue?): String =
when (color) {
    is HexaColor -> color.toPKM()
    is RgbColor -> color.toPKM()
    is HslColor -> color.toPKM()
    is BaseColor -> color.name
    null -> "#000000"
    else -> "#000000"
}


private fun cleanStr(s: String): String =
    s.replace("\"", "")
        .replace("😀", "@[:smile:]")
        .replace("😢", "@[:sad:]")
        .replace("😐", "@[:serious:]")
        .replace("❤️", "@[:heart:]")
        .replace("⭐", "@[:star:]")
        .replace("😸", "@[:cat:]")
        .replace("😺", "@[:^^:]")
        .trim()

// Funciones de utilidad
fun serializeElement(element: Any): String =
serializarElemento(element)
fun serializeSection(element: SectionsModel):
String = serializarSeccion(element)
fun serializeTable(table: TableModel): String =
serializarTabla(table)
fun serializeStyles(styles: List<Any>?): String =
serializarEstilos(styles)
fun serializeColorValue(color: ColorValue?):
String = serializarColor(color)
```